A Comparison of Modelling Methods: Customer Order-In Propensity

Shahrukh Kamal Khan - 000998261

Supervisor: **Christopher Walshaw**

Final Year Individual Project COMP1682

A dissertation submitted in partial fulfilment of the University of Greenwich undergraduate degree programme

BSc (Hons) Computer Science

11/05/2021

Word count: **9430 words**

ABSTRACT

The final year project is based upon building and experimenting machine learning models to calculate and predict customer Order-in propensities and Collection propensities. This report compares Linear Regression, LASSO, Ridge, Random Forest Regressor and XGBoost Regressor models to predict propensity of order-in and collection at Argos stores on a sample of 1,000,000 rows of randomly selected data. Data pre-processing, hyperparameter tuning of models and experimentation with different combinations of attributes are discussed in this report. The final model uses XGBoost as it performs better than the rest with RMSE value of 0.3631 for predicting order-in propensities. The model accuracy decreases and error increases when predicting collection propensities. It is observed that including demand units in training data as input increases the models performance.

ACKNOWLEDGEMENTS

I would like to express sincere gratitude to Argos Data Science team and the University of Greenwich for the Knowledge Transfer Program which enabled us to study retail data and apply machine learning and data science knowledge on an excellent project and case study. Many thanks to Christopher Walshaw and Laszlo Torjai who provided critical feedback and guidance that helped me significantly improve the project and the dissertation itself. I would also like to thank the Computer Science Faculty at the University of Greenwich for their efforts in educating and teaching us numerous things not just about the course but also professional life throughout the years.

The past year has been hard for everyone in the world and I am personally grateful for everyone's effort's in helping me succeed in life till this point and beyond.

Contents

1 INTRODUCTION

Retail businesses, big or small, use machine learning techniques to support different business operations or learn trends in customer behaviour. Real time chat-bots using NLP, Decision Support to analyse different scenarios, Customer Recommendation Systems and Churn Predictions are some of the examples. Different problems and aspects are tackled through different machine learning algorithms and data analysis. Similarly, Argos has a humongous catalogue offering more than 60k products (Argos, 2019), whilst offering 20-30k products in store for collection there is always a need for smart inventory management plan. This serves as a difficult problem due to having thousands of stores around the UK in distinct locations , simultaneously also having to predict different aspects of numerous products with a customer point of view (Walshaw, 2020). Argos parent company Sainsbury recently started working in with partnership with Accenture to build machine learning solutions on Google Cloud Platform to gain new insights about customers and product trends. They have been able to develop predictive analytic models to spot trends and adjust price range for a better customer experience (Coad, 2019).

The problem at hand for retail businesses is to manage the different variety of products in stores i.e. to decide what products to keep in store to maximize profit and customer satisfaction. While this can be tackled in many ways e.g. finding out best products through sales records or demand of a product, however this would decrease the variety of products sold at a store and result in loss of sales and profits due to lack of low demand products at store. A sensible way to tackle this problem is by calculating and predicting “product order in propensity”. Propensity here refers to the probability of or inclination towards a product being “ordered in” at a store. Higher propensity means higher chances of a product being ordered. The idea here is that products with higher propensities are being ordered frequently by customers hence it would be counterproductive to keep them at the store. Products with lower chances of being ordered would be kept in store to convert the potential loss into profit if the product was not in store.

The primary objective of this report is to build a reasonably good data driven model for future predictions. The study is carried out in collaboration with Argos, one of the leading retail firms in the UK, on data provided by the firm from the past 3-3.5 years (2017- 2020) including product, location and calendar specific data. The data consists of demand, order and collection units and various store and product related variables. All the instances have a minimum of one demand unit whereas the order and collection units vary. This report tends to provide an overall machine learning solution to predict order in propensities and collection propensities (probability of a product being collected with respect to its demand) for products at a certain date and store location. Data analysis performed on the dataset is discussed further in the paper and intuitions made due to analysis which lead to feature engineering and selection for machine learning models. Problems faced during the project and their solutions are

also elaborated in separate sections of the paper. Machine learning models namely Linear Regression, Lasso, Ridge, Random Forests Regressor and XGBoost are trained and evaluated on the pre-processed data. The results are then debated upon to select the best machine learning model for predictions.

2 LITERATURE REVIEW

2.1 Introduction

Artificial intelligence and big data has disruptively changed almost all the industries ranging from telecommunication, health services, medicine and finance, due to the need of prediction models for various problems, making its inclusion a necessity in every business (Santis, et al., 2017). With the continuous increase in unstructured and stream data, it is still a challenge for researchers to transform it into meaningful data. McKinsey suggests that companies that use data and business analytics to make decisions are more productive and deliver higher returns on equity than companies that don't (Manyika, et al., 2011). Machine learning provides automated methods of data analysis which enables us to predict future data by spotting patterns in historical data and performing other decision making (Murphy, 2012). Supervised Learning, a sub-category of machine learning, is used to classify data or predict outcome based on labelled historical data accurately. Supervised learning is being used by different organizations to tackle variety of diverse problems ranging from spam filtering, cancer detection to predicting house and stock prices. The category can be halved further into, namely classification and regression. Classification algorithms, as the name suggests, are used to classify data into specific classes by recognizing features or entities to conclude how a class should be defined. Whereas Regression algorithms provide an understanding of dependant and independent variables in a dataset to form up a hypothesis to predict continuous data. Contrary to Supervised Learning, Unsupervised learning is used to analyse previously unknown data to spot trends and group similar data points in a dataset. The algorithms are used to discover hidden patterns and give insights which can be used to draw conclusion to solve various problems. It is used in exploratory data analysis, customer segmentation, image and object recognition. Clustering, Association Rules and Dimensionality Reduction are some of the common approaches in Unsupervised Learning (IBM Cloud Education, 2020).

Businesses use predictive statistical analysis to build propensity models that predict customers who will buy their product. This enables the business to focus on specific customers not all, and to approach them with a customized plan that would yield a higher chance of success rather than approaching every known customer resulting in waste of time and resources, and would not be as fruitful. Predicting customer churn is one of the problems faced by many organizations i.e. telecommunication companies, retail and banking firms. Business use churn models to create better products, improve services and customer experiences to achieve high customer retention.

Classification algorithms like decision forest, logistic regression, neural networks and support vector machines are some of the machine learning techniques commonly used to create churn models (Barga, et al., 2015).

2.2 Problem Description

With a wide variety of products offered by Argos for collection from stores, it is not possible to accommodate all the products at every store due to size of stores. Only a few stores have the physical capacity to hold the stock of all the products on sale by the company. However, the customers have an option to "order in" their required product at their nearest store if it is not already available. The physically smallest level of store or the normal stores that we have around us are called spokes which are linked to bigger stores that also act as a support for the spokes to hold stock inside them. These are called hubs. Furthermore the hubs are linked to regional fulfilment centre's which are linked to distributional centres. The ordered products arrive in 24hrs time span from the linked hub if the hub has the product. Not all customers choose to order the products and wait one day instead they buy it from another brand which results as a loss of a customer and possible profit for the company. Argos wants to have an insight to this customer behaviour, to predict the probability of customers choosing to order various products enabling the company to know what products to keep at small stores for sale which ideally would be the ones having a lower chance of being ordered whilst having a good demand.

Churn prediction is somewhat similar to our problem although we do not want to predict the customers labels, rather we want to predict the propensity which is a continuous numerical variable (SimpleOrderPropensity or CollectOrderPropensity) hence regression models are used.

The research topic of calculating and predicting product order in propensities seems to be one of a kind. There is no research available that has been conducted prior, that is directly linked to our topic. Although it is speculated that companies might already be working on it or if already done then it has not been disclosed due to various reasons, as stated by Laszlo Torjai (Senior Data Analyst at Argos). Research has been conducted on predicting different aspects in a retail point of view like redeeming coupons, or time series analysis and forecasting for demand and sales.

2.3 Related Work

2.3.1 Industrial Overview

Investments in cognitive systems and artificial intelligence frameworks will increase to $77.6 billion by 2022, more than three times the $24 billion estimate for 2018 as per the (IDC, 2018). The banking and retail businesses has reportedly made the biggest investments in cognitive and artificial intelligence frameworks in 2018. A survey by (O'Reilly Media, 2018) reports that 5,500 of the world's 11,000 data science experts work for firms that are in the beginning phases of investigating machine learning for the challenges faced by the firms, while the rest are extensively experienced in creating

and deploying end to end machine learning solutions. In 2019 the most advertised job posts were of machine learning specifically machine learning engineer in the United States, reports (Indeed.com, 2019).

The banking sector constantly utilises, machine learning applications searching out the best procedures for investments and automating financial procedures. JPMorgan has created a qualified machine learning model to track down the best execution technique for trading orders. Utilizing past data and calculations based on it, the model figures the likelihood of the best performing trade orders and executes them likewise. It currently delivers suggestions to human traders, however is actively working as an automated trader and performing transactions (McDowell, 2018). Toronto Dominion Bank as well has invested its resources into machine learning based portfolio management models to examine through the stocks to find ones with the most anticipated volatility, and it at that point utilizes a traditional risk method to make low-volatility portfolios. This diminishes the ex post volatility more than existing risk evaluation methods (Bodjov, 2018). Machine learning also proves to be benificial in preventive maintainance. It allows manufacturers to understand various aspects through data generated by machines and to improve predictive models over the long run. General Electric (GE) utilizes machine learning models to distinguish anomalies and patterns in machines they use, they then build up a model for their machines, and predict when machines will require maintainance (Forbes, 2017). At D&C, machine learning facilitates agricultural equipment, and helps predict and identify plants that might be in need of pesticides, fertilizers or other chemicals depending on its situation and only providing optimal amount of chemicals when actually required (Grosch, 2018).

2.3.2 Linear Regression Models

Linear Regression can be used to predict continuous target variables with one or more predictor variable which is known as Multiple Linear Regression. Linear Regression models make numerous assumptions in regards to the distribution of the data being used, which can't generally be satisfied in real life scenarios as the data cannot perfectly align with the assumptions. The data should be normally distributed, while the variables being homogeneous , and that they are not directly dependant on each other or auto correlated (Vittinghoff, et al., 2005).

Researchers have made use of regression models such as Linear Regression Model, Support Vector Machines and Gradient Boosting Decision Tree based algorithms such as XGBoost for regression problems such as predicting sales or house prices. (Fan, et al., 2018) makes a good use of regression models namely as Lasso, Ridge, SVR and XGB for predicting housing prices in the Ames Dataset for kaggle prediction competition. The linear models Lasso and Ridge utalise the preprocessed data optimaly and give lowest Average MSE values. Ridge model performs best with an alpha parameter of 10. Similarly, L. Lui, Emil Janulewicz, Nissan Pow also used Linear Regression, SVR, KNN and Random Forest models to predict property sale price. An ensemble model comprising of KNN and

Random Forest was also used which performed better than the rest with an error of 0.0985 (Pow, et al., 2014).

2.3.3 Ridge Regression Models

Ridge Regression model rose in popularity shortly after a publication by Arthur E. Hoerl & Robert W. Kennard (Hoerl & Kennard, 1970), since then it has been widely used and also adapted into the alternative Lasso and Elastic Net models. Ridge Regression is said to have been used in applied statistics to deal with collinearity problems in general least square regression models (Charnes, et al., 1986). It tends to deal with the problem of collinearity without removing the variables. This characteristic proved to be important in some application as showcased by McDonald and Schwing (Donald & Schwing, 1973), where they discuss mortality rate in relation to air pollution and instabilities in predicions from regression models. It proposes that ridge regression model presented by Arthur E. Hoerl & Robert W. Kennard achieves stable coefficients when compared with Ordinary Least Square model. Ridge regression is similar to LASSO however uses L2 regularization (2).

$$L_1 = \Sigma_{i=1}^{n} |y_{true} - y_{pred}| \tag{1}$$

$$L_2 = \Sigma_{i=1}^{n} (y_{true} - y_{pred})^2 \tag{2}$$

2.3.4 LASSO Regression Models

The Least Absolute Shrinkage and Selection Operator or LASSO model is an estimation model that minimizes the error and acts as a feature selector by minimizing the weights close to zero therefore neglecting least useful features. Lasso can be applied in various statistical models as stated by Robert. Standard regression models tend to overfit in terms of the variables used to train them, and can provide an overestimation of the model perofrmance. LASSO uses regularization/penalization to avoid this problem (Tibshirani, 1996). LASSO regression models use L1 regularization (1) on the cost function. It identifies variables with lesser weights resulting into minimizing the models prediction error. Alternatives to Lasso are Ridge and Elastic Net that use different regularization methods (Ranstam & Cook, 2018). There are often problems of selecting useful feature variables from numerous different possibilities in marketing analytics as retail store sales are affected by inter and intra category characteristics. (Ma, et al., 2016) makes use of LASSO regression to identify key and influential categories and rolling scheme to generate sales forecast. The method proves to be successful as it demonstrates improvements in forecast accuracy. Likewise, (Martinez, et al., 2020) use LASSO as one of the methods to develop machine learning solution for predicting future sales in a non-constructural setting and (Bertsimas, et al., 2016) also uses LASSO in its study to predict outcomes of clinical trials.

2.3.5 Random Forest and XGBoost Models

A Random Forest is a group of Decision tree predictors, an example of an ensemble, that trains via bagging method. The algorithm makes use of all the cores available training trees in parallel. Many machine learning methods have been developed and used, however Boosting models including Gradient Tree Boosting regularly prove to be robust and perform the best for different applications. Boosting tends to imporve accuracy for any given algorithm (Freund & Schapire, 1999). Boosting method combines results of several weak classifiers or predictors which averages the predictions (in regression context) and provides increased accuracy. It is an ensemble method that trains predictors sequentially where each predictor tries to reduces its predecessors errors. The two most popular Boosting methods are Ada Boosting (Adaptive Boosting) and Gradient Boosting (Friedman, 2001). In our work we use the Gradient Tree Boosting algorithm known as Extreme Gradient Boosting (XGBoost). Gradient Tree Boosting technique has performed well in many applications. Tree boosting has proved to provide state-of-the-art results on many classification benchmarks (Li, 2010). The model is also used in real world production pipelines for ad click through rate prediction (He, et al., 2014). The importance of this model has be evident as it has been extensively recognized in numerous machine learning and data mining challenges. In 2015, amongst 29 machine learning challenges hosted by Kaggle 17 winning teams used XGBoost models out of which eight were used alone as a model and others were combined with neural networks in ensembles as mentioned in a blog post by Kaggle. In KDDCup 2015 all the top 10 winning teams used XGBoost, and reported that only a small ensemble methods only slightly performed better in certain occasions, (Chen & Guestrin, 2016). The challenges mentioned above were predicting store sales, high energy physics even classification, web text classification, predicting customer behavior, motion detection, ad click through, hazard risk prediction, online course droupout rate prediction. XGBoost performed well for all the mentioned tasks even though data analysis, feature engineering and domain expertise also played an essential role, the model's importance is evident. One of the reasons of its success is its scalability. XGBoost has been described as a sclable machine learning system. The system runs 10x faster than other models and scales to billion of instances in memory limited settings, (Chen & Guestrin, 2016).

2.3.6 Various Factors and Their Impacts on Consumer Behaviour in Retail

Emperical studies have been carried out on product returns as earliest as 1997 by (Hess & Mayhew, 1997) providing a model to predict return timings for appearl marketer. (Janakiraman, et al., 2016) suggest that return policies can increase sales as compared to returns. Lincence in return policies majorly contribute to this effect. (Anderson, et al., 2009) developed a structural model of a customer's decision of purchasing and returning a product. The model when applied to the data of a mail-order catalouge company exhibited significant variation in return values across different customers and categories. The model illustrates how return policies can be optimized by retailers for different products and categories. Our study focuses to extract information relating to orders and demands, the

research above indicates that good return policies can have a good effect on product sales and demands.

The weather conditions are also believed to have an essential role in product sales and demands. Numerous researchers, upon analysing weather data in combination with commercial data have found that weather is one of the factors affecting economic and commercial behaviour. (Kang, et al., 2010) investigates the climate consequences for returns as well as instability in the Shanghai stock exchange. Extreme climate condition dummy data is produced by utilizing the 21-day and 31-day moving average and its standard deviation. It states that the climatic conditions affect the two types of shares differently while having a strong effect on B-share returns only. It is proposed that the impact of climate and, specifically daylight is positively related to consumer spending's. People tend to consume or purchase more with increasing daylight, (Murray, et al., 2010). Another group of researchers (Badorf & Hoberg, 2020) study the influence of weather on daily sales in B&M retailing examining empirical data for 673 stores. The paper states that including the weather forecast in sales prediction can improve the accuracy for up-to seven days. Although, the improvement of forecast accuracy diminishes for a higher forecast horizon. Weather data is also applied to avocado sales data to estimate a sales forecast for avocado's using machine learning models, (Rincon-Patino, et al., 2018).

3 REQUIREMENTS ANALYSIS

Argos provided a set of historical input data including data for the product x store x day level components of the target variable. The project evaluates different machine learning modelling methods namely Linear Regression, LASSO, Ridge, Random Forest Regressor, and XGBoost Regressor that can use the input data to train a model to predict the target. The report includes a recommendation for a model that Argos can implement to use for predicting customer order-in propensity.

3.1 Methodology

The experimentation process follows few basic steps, typically used in data science problems.

3.1.1 Defining the business problem

It is essential to know about the problem in depth, so that it can be further broken down for the ease of solution or to know how to formulate a plan for the solution. The project problem is elaborated in the Introduction and Literature Review section.

3.1.2 Data collection and preparation

Data can be collected through data mining although there is a wide range of sets of data available to public on the internet. Data can be loaded to IDEs in the form of CSV or Excel files. The data is then

cleaned to fill up or remove any missing values. Data for this project is collected and prepared by Argos data science team, provided to us. The data is in 4 parts and need to be merged according to the relevant columns in each data set and irrelevant columns (variables needed for other projects) need to be discarded. Data does not have any missing values however the dataset is large and takes a lot of memory.

3.1.3 Data analysis

It is important to visualize the data to gain insights and develop an understanding about the data. The knowledge then allows us to decide what model development approach to take. Data can be visualized by creating scatter plots, box plots, histograms and more complex graphs depending on the data and its use case. Machine learning algorithms benefit from standardization of dataset, so we fix outliers or/and different scales fields in our datasets. Using pre-processing package, we change raw feature vectors into suitable form of vector for modelling. Detailed data analysis is done in further sections describing various aspects of the datasets.

3.1.4 Feature selection

Crucial part of data pre-processing is feature/variable selection. The process is to find the right variables for predictive model which is critical in large datasets involving lots of data variables. Using too many variable is useless as it would waste resources and the model would turn out to be extremely specific to the data while overfitting. The model will fail to predict any new data other than the current data set. Different correlation methods like Pearson, Kendall and Spearman are used to find essential features from a dataset. After applying Pearson's correlation on the dataset with the target variables it shows that the features have low linear correlation to the targets.

```
Date                       0.166388
ProductKey                -0.000888
LocationKey                0.000433
DemandUnits               -0.013628
OrderedUnits               0.545383
CollectedUnits             0.504529
YearWeek                   0.176053
DayOfWeek                 -0.004923
IsBankHoliday             -0.004925
IsWorkingDay               0.012464
HierarchyLevel1           -0.027593
HierarchyLevel2           -0.029107
Seasonal                   0.011643
IsHub                     -0.044916
LocationType1              0.113934
LocationType2              0.061378
Latitude                  -0.007309
Longitude                  0.048783
SimpleOrderPropensity      1.000000
CollectOrderPropensity     0.634519
Name: SimpleOrderPropensity, dtype: float64
```

1: Correlation to SimpleOrderPropensity

```
Date                       0.195762
ProductKey                -0.000264
LocationKey                0.001233
DemandUnits               -0.039637
OrderedUnits               0.319641
CollectedUnits             0.795272
YearWeek                   0.202194
DayOfWeek                 -0.003448
IsBankHoliday             -0.015933
IsWorkingDay               0.020220
HierarchyLevel1           -0.020989
HierarchyLevel2           -0.021353
Seasonal                   0.023115
IsHub                     -0.056929
LocationType1              0.153551
LocationType2              0.074514
Latitude                  -0.006158
Longitude                  0.029045
SimpleOrderPropensity      0.634519
CollectOrderPropensity     1.000000
Name: CollectOrderPropensity, dtype: float64
```

1.2 Correlation to CollectOrderPropensity

3.1.5 Model Development

Firstly we have to choose the right algorithm for our problem. One of the tasks of the project is to use and analyse different algorithms to conclude which wo be the best approach towards the solution of this problem. Regression algorithms are researched upon as they fit the problem domain best. The problem is to predict what number of times a product would be ordered or collected in relation to its demand. Research from the Literature Section suggests use of the regression algorithms described above. We split our dataset into train and test sets to train the model, and then test the models accuracy separately. After that we can set up our algorithm and train our model with the train set. We use our test set to run predictions and the result tells us what the class of each unknow value is. We use different metrics to evaluate our model's accuracy.

3.1.6 Model testing and validation

After the model is trained we have to test it using sample of the dataset that we did not use in training. The sample can vary from 20-30% of the whole data. We select a portion of the data for training and the rest is used for testing. The model is built upon training data then the test feature set is passed to the model for prediction. The predicted values for the test set are compared with the actual values. This approach gives us a better evaluation on out of sample accuracy however it highly depends on which dataset the model is trained and tested upon.

K-fold Cross Validation is an approach which resolves the issues of dependency and low out of sample accuracy. The data is divided into four folds where in each fold of the data is set as test data and the model is trained on the rest. This continues for the other folds changing the test data, whilst calculating the out of sample accuracy for each fold. Finally, the accuracy is averaged to get a correct evaluation.

The next sections elaborate various model scores and accuracies.

3.1.7 Model performance

Evaluation metrics are used to explain the performance of a model. We can compare the predicted values with the actual values to calculate the accuracy of a model. Evaluation metrics provides us an insight to areas that require improvement. Some of the Evaluation Metrics used for Regression Models are Mean Absolute Error, Mean Squared Error and Root Mean Squared Error. Error is the difference between the data points and the trend line generated by the algorithm. We use Root Mean Squared and Mean Absolute Errors to judge the predictions made by the models in this report.

3.2 Project Environment and Libraries

The project is based upon python programming language due to its flexibility and ease of use. There are various libraries that are used for different tasks of machine learning from splitting the dataset to training models, some of which are as follows. Free Kaggle notebooks have been used to

carry out tasks briefed in the methodology sub-section above. The tasks are further explained in data analysis and model development sections. Kaggle.com provide free notebook kernel's with 16gb of ram and 19.6gb of disk space. Kaggle also provides accelerators like TPU and GPU but are not needed for our project due to the nature of the case stud problem. Other free notebooks like Google CoLab only provide 12.7gb of memory however it provides more than a 100gb of disk space and data can be easily imported from google drive but needs to authenticate when linking the drive.

NumPy: Math library to work with N-dimensional arrays in python. It is effective and efficient for computation (NumPy, 2021).

Matplotlib: Plotting package allowing 2d and 3d plotting (Matplotlib.org, 2021).

Pandas: Library providing high-performance and easy to use data structures. Many functions for data importing, manipulation and analysis. It provides data structures and operations for manipulating numerical tables and timeseries (pydata.org, 2021).

Dask: Library for parallel computing in python to deal with big data. The library extends numpy and pandas and has similar functions to them (Dask.org, 2021).

SciKit Learn (sklearn): Collection of algorithms and tools for machine learning. It has most of the classification, regression and clustering algorithms. Works with NumPy and SciPy. It has great documentation and is easy to implement. Most of the tasks in the machine learning pipeline are already provided as function in the library (sklearn.org, 2021).

4 LEGAL, SOCIAL, ETHICAL AND PROFESSIONAL ISSUES

Machine Learning models are trained upon data collected in various ways. The data is an important part of the machine learning development process as it makes predictions based on it, hence organizations and firms need to take into account the legal data rights. The data rights might differ in certain situations such as type of data, who it belongs to or the use case of the data. Violation of the rights or ethical implications can result into serious penalties towards individuals or group of people (Mittelstadt., et al., 2016). Ethical issues about machine learning applications may emerge at any stage of the application development from data collection to model deployment. During model, Machine learning algorithms learn from the data without having the ability to assess the data like humans , their learning is basically tuning of variable weights and different calculations hence they display an bias if the data itself is biased (e.g., sex, race/religion, or age). The end users should be prevented from the bias for their use case result. For our project the data has been collected by Argos from its retail stores around the UK. The data does not have any variables related to any pupil rather it has details of products, store locations etc as discussed in previous sections. The data has only been provided to us for educational purposes legally, and we cannot share or sell the data to a third party. Although the data has been redacted and exact details of store locations and products have been encoded.

5 DATA ANALYSIS

Data Analysis is an essential part of developing a machine learning solution as it allows us to gather information and various trends from our data, through which we can comprehend the business problem and plan on how to use the data. The process helps us to decide how to use machine learning models and what steps to take in order to have a better overall result.

5.1 Dataset Description

The data comprises of different product, location and date characteristics including with Demands, Orders and Collection of specific products on specific dates and at specific store locations. The data was in 4 parts hence it was merged to get all the characteristics together. The merged dataset has 43,016,599 rows and 20 columns including the propensity columns (SimpleOrderPropensity and CollectOrderPropensity). The table below shows all the variables present in the dataset.

Variable Name	**dType**	**Value**	**Description**
Date	int64	Date	Date in numeric format YYYYMMDD
ProductKey	int64	Identifier	Unique product identifier
LocationKey	int64	Identifier	Unique location identifier
DemandUnits	int64	Continuous	Number of products demanded
OrderedUnits	int64	Continuous	Number of products ordered
CollectedUnits	int64	Continuous	Number of products collect
YearWeek	int64	Ordinal	Week indentifier in the format YYYYWW, where WW denotes the week sequence within the year
DayOfWeek	int64	Ordinal	Sunday = 1, Saturday = 7
IsBankHoliday	bool	Nominal	Bank holidays denoted by 1, otherwise 0
IsWorkingDay	bool	Nominal	Normal working days denoted by 1, otherwise 0
HierarchyLevel1	int64	Ordinal	Product hierarchy by type of products. Examples of this hierarchy level include Kettles, Televisions, Bedsheets
HierarchyLevel2	int64	Ordinal	HierarchyLevel1 maps many-to-one to HierarchyLevel2. Examples of this hierarchy level include Household Electricals, Technology, Furniture
Seasonal	bool	Nominal	1 for products that have a distinct "on" season, eg. garden furniture (summer) and fan heaters (winter); 0 otherwise. Demand is typically more

			volatile on seasonal products so forecast volatility may be higher
IsHub	bool	Nominal	A "hub" means location that suppliers stock to other locations as well as to its own direct customers
LocationType1	int64	Ordinal	Classifies locations by their shop-front type. Examples include High Street, Retail Park, Within Supermarket
LocationType2	int64	Nominal	Classifies locations by their surrounding area. Examples include Town Centre, Rural, Destination Shopping Centre
Latitude	float64	Discrete	Latitude north of the equator
Longitude	float64	Discrete	Longitude east of the meridian
SimpleOrderPropensity	float64	Continuous	"Order-in" probability of a product with respect to its demand.
CollectOrderPropensity	float64	Continuous	Collection probability of a product with respect to its demand

As mentioned above the dataset is large and it took a lot of memory when all the datasets were loaded into the memory and merged together. It was not possible to load, merge and save .csv file of the merged dataset with 8gb of ram (system specs: core i5 8gb ram). The merge was performed using dask library where the data was loaded partially and only when a computation was performed using .compute() function. The total data uses 5.3gb of memory space.

```
dtypes: bool(4), float64(4), int64(12)
memory usage: 5.3 GB
```

The propensities were calculated using the formulas shown below:

$$SimpleOrderPropensity = \frac{OrderedUnits}{DemandUnits}$$

$$CollectOrderPropensity = \frac{CollectedUnits}{DemandUnits}$$

Dataset rows, columns, variables, numeric variables vs categorical variables vs identifying variables.. target variable. Summary (mean/median/mode) for numeric variables…

5.2 Exploratory Data Analysis

EDA is essential in machine learning and data science as it gives important insights about the data and the model to be developed. It is useful to know certain characteristics of the variable that will be used

to build a machine learning model. Using pandas dataframe's describe function the mean, median, quartiles and minimum and maximum values are output.

```
pd.set_option('display.float_format', lambda x: '%.2f' % x)
main_df.loc[:, ["DemandUnits", "OrderedUnits", "CollectedUnits",
                "SimpleOrderPropensity", "CollectOrderPropensity"]].describe()
```

	DemandUnits	OrderedUnits	CollectedUnits	SimpleOrderPropensity	CollectOrderPropensity
count	43016599.00	43016599.00	43016599.00	43016599.00	43016599.00
mean	1.14	0.91	0.69	0.80	0.62
std	0.70	0.78	0.65	0.39	0.48
min	1.00	0.00	0.00	0.00	0.00
25%	1.00	1.00	0.00	1.00	0.00
50%	1.00	1.00	1.00	1.00	1.00
75%	1.00	1.00	1.00	1.00	1.00
max	580.00	580.00	294.00	4.00	3.33

It can be seen that average (mean) demand of units is greater than average orders, and that greater than average units collection. Most of the values in the propensities and Demand, Order and Collection units columns is "1" as observed in the quartile values (25% 50% 75%) however the max values exceed more than 500 times more than the values ("1") that the dataset mostly comprises of.

- There have been 65,160 different type of products in Argos inventory being sold across 1,247 stores locations in 3 years' time span!

```
main_df.ProductKey.value_counts()
```

```
28152    16900
63886    15916
61980    15879
55943    15516
42283    15190
         ...
72782        1
1932         1
34714        1
67495        1
76894        1
Name: ProductKey, Length: 65160, dtype: int64
```

```
main_df.LocationKey.value_counts()
```

```
25194    195916
72448    166560
26800    155710
67378    152407
39596    146126
          ...
21726        21
58455        20
14920        15
60269        15
1873         11
Name: LocationKey, Length: 1247, dtype: int64
```

- There are 181 different types of products.

```
len(main_df["HierarchyLevel1"].value_counts())
```

```
181
```

- HierarchyLevel1 maps many-to-one to HierarchyLevel2. The 181 different types of products can be grouped into 26 main types.

```
len(main_df["HierarchyLevel2"].value_counts())
```

```
26
```

- There are seven different shop front types of the Argos stores. Examples include High Street, Retail Park, Within Supermarket

```
len(main_df["LocationType1"].value_counts())
```

```
7
```

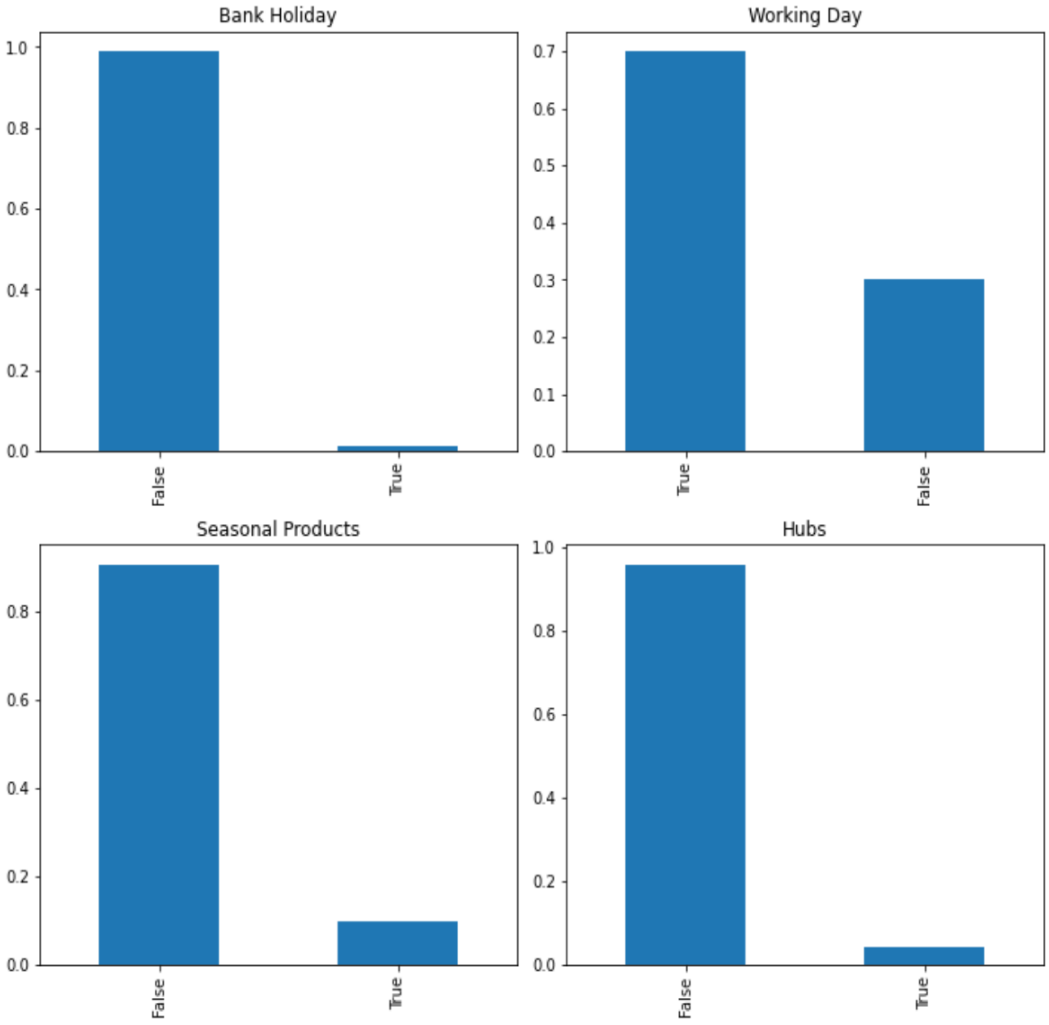
- Argos stores can be distinguished by three distinct surroundings.

```
len(main_df["LocationType2"].value_counts())
```

```
3
```

5.2.1 Graph Plots for Nominal Variables

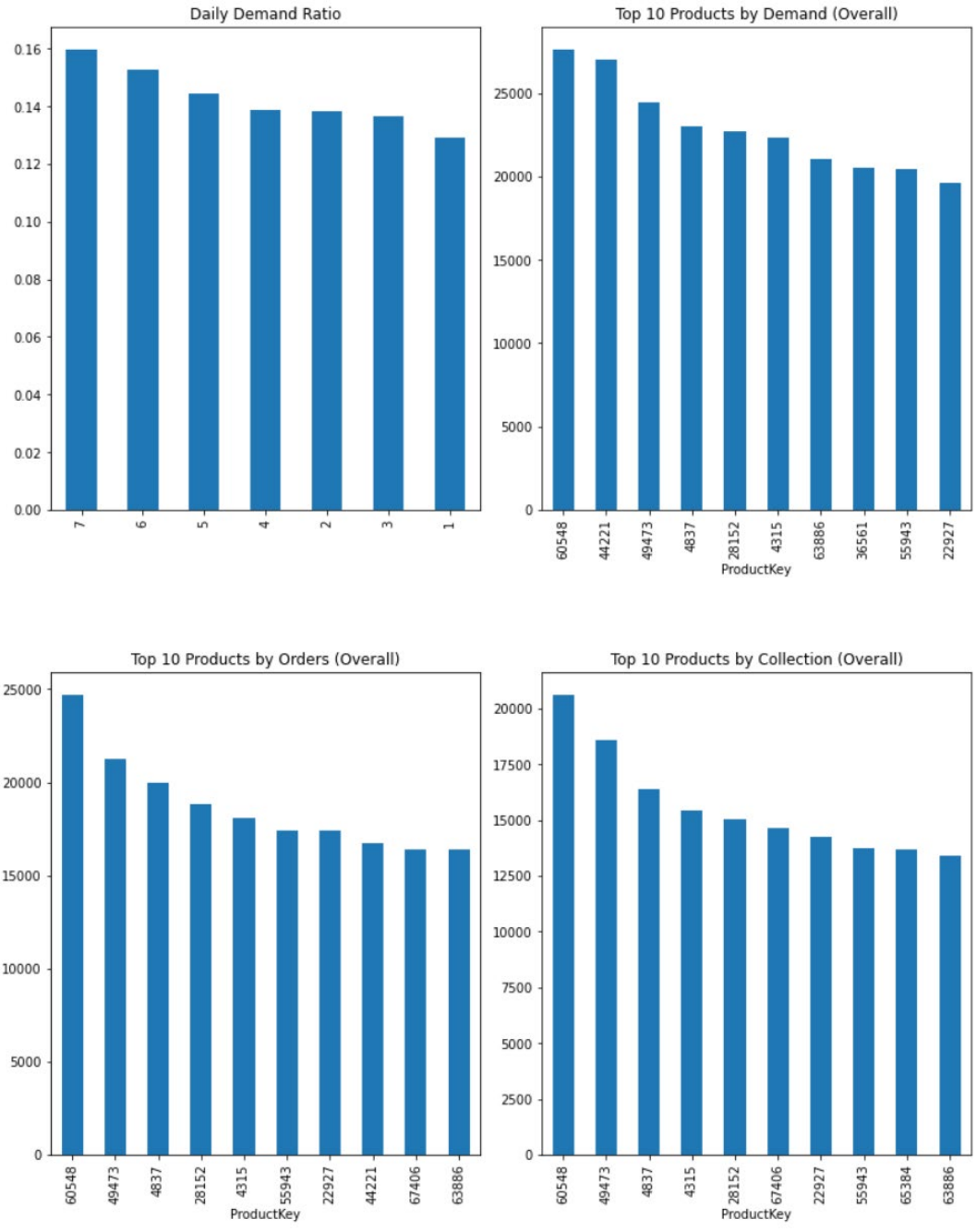
```
plt.subplot(321)
main_df.IsBankHoliday.value_counts(normalize=True).plot(kind="bar", title="Bank Holiday", figsize=(10,13))
plt.tight_layout(pad=1)
plt.subplot(322)
main_df.IsWorkingDay.value_counts(normalize=True).plot(kind="bar", title="Working Day")
plt.tight_layout(pad=1)
plt.subplot(323)
main_df.Seasonal.value_counts(normalize=True).plot(kind="bar", title="Seasonal Products")
plt.tight_layout(pad=1)
plt.subplot(324)
main_df.IsHub.value_counts(normalize=True).plot(kind="bar", title="Hubs")
plt.tight_layout(pad=1)
```



The top left corner graph show that only approximately 2% of the data instances are True for bank holidays. This suggests that product demands have been mostly made on non-bank holiday's, however there have been only been 24-32 bank holidays during the 3-4 year time span of the data (GOV.UK, 2021). About 70% of the instances are of working days and there are 5 working days in a week hence it makes sense. 88% of the demands have been there for non-seasonal products however we do not know what different seasons are. Approximately 96% of the data instances have been true for spokes (false for IsHub), hence it can either be that people are not ordering products from hubs or they already find their desired products at the hubs therefore no need to order.

5.2.2 Graphs Plots for product related statistics

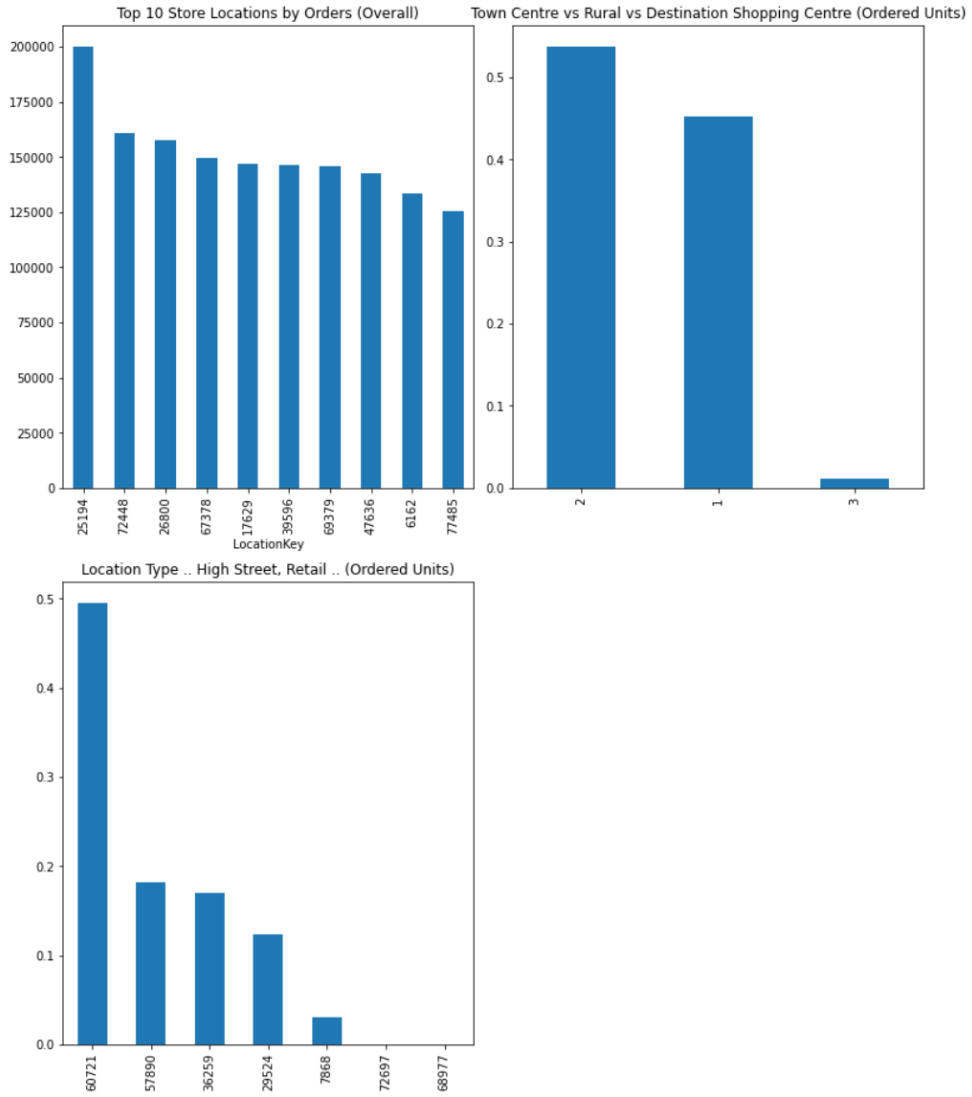
```
plt.subplot(221)
main_df.DayOfWeek.value_counts(normalize=True).plot(kind="bar",title="Daily Demand Ratio",figsize=(11,13))
plt.tight_layout(pad=1)
plt.subplot(222)
main_df.groupby("ProductKey").DemandUnits.sum().sort_values(ascending=False).head(10).plot(
    kind="bar",title="Top 10 Products by Demand (Overall)")
plt.tight_layout(pad=1)
plt.subplot(223)
main_df.groupby("ProductKey").OrderedUnits.sum().sort_values(ascending=False).head(10).plot(
    kind="bar",title="Top 10 Products by Orders (Overall)")
plt.tight_layout(pad=1)
plt.subplot(224)
main_df.groupby("ProductKey").CollectedUnits.sum().sort_values(ascending=False).head(10).plot(
    kind="bar",title="Top 10 Products by Collection (Overall)")
plt.tight_layout(pad=1)
```



The daily demand ratio graph shows the ratio of instances being specific days from 1-7. The week starts with Sunday being the first day. Sunday:1, Monday:2, Tuesday:3, Wednesday:4, Thursday:5, Friday:6, Saturday:7. The most order in's and demands for products are on Saturday 16% and Friday 15.6% whereas Sunday being the least at approx. 13% which suggests people would not want to order or collect products on a Sunday as it the end of the week. ProductKey 60548 has been demanded, ordered and collected the most throughout the timespan however the overall collection is lesser than the orders and that is lower than the demand itself. ProductKey 44221 has had the second highest demand however it had the 8th highest orders and is not even in the top 10 products for collection. This product might be a prime example of our case study hypothesis as this product would have a low order in and collection propensity but has a great demand therefore should be kept in store so that customers can buy it at the exact time rather than ordering. Similarly ProductKey 63886 is on the 7th position for highest demands however has the least orders and collections amongst the top 10 products.

5.2.3 Graph Plots for Location specific statistics

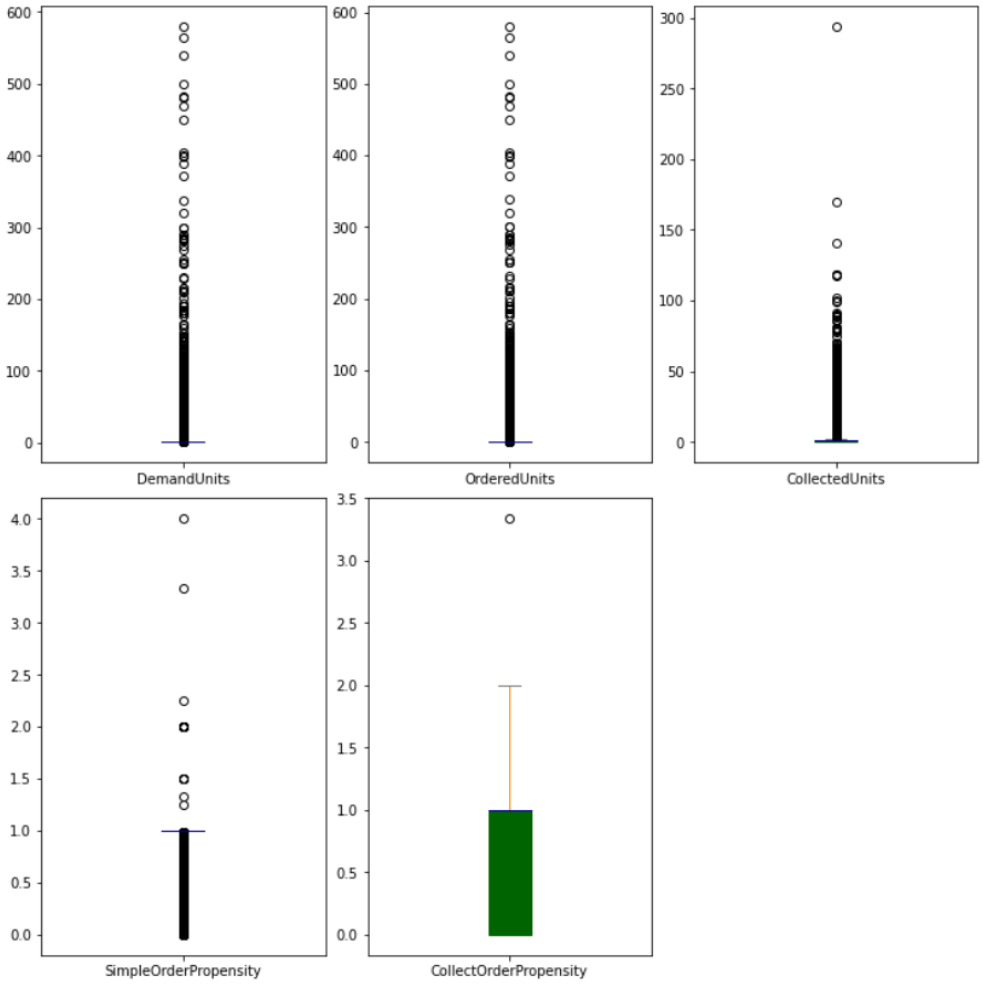
```
plt.subplot(221)
main_df.groupby("LocationKey").OrderedUnits.sum().sort_values(ascending = False).head(10).plot(
    kind="bar", title="Top 10 Store Locations by Orders (Overall)", figsize=(11,13))
plt.tight_layout(pad=1)
plt.subplot(222)
main_df.LocationType2.value_counts(normalize = True).plot(
    kind="bar", title="Town Centre vs Rural vs Destination Shopping Centre")
plt.tight_layout(pad=1)
plt.subplot(223)
main_df.LocationType1.value_counts(normalize = True).plot(
    kind="bar", title="Location Type .. High Street, Retail Park ..")
plt.tight_layout(pad=1)
```



LocationKey 25194 has had the most orders reaching up to 200,000 orders almost 50,000 more than the store in second place. LocationType2 value 2 store locations has occurred the most in the dataset at 54% where as value 1 store location are 44% of the dataset and LocationType2 value 3 is only 2%. LocationType1 68977 and 72697 are almost non-existent. These type of stores might be one time pop ups or experimental stores.

5.2.4 Box Plots for Continuous Variables

```
plt.subplot(231)
main_df["DemandUnits"].plot.box(figsize=(10,10))
plt.tight_layout(pad=0.5)
plt.subplot(232)
main_df["OrderedUnits"].plot.box()
plt.tight_layout(pad=0.5)
plt.subplot(233)
main_df["CollectedUnits"].plot.box()
plt.tight_layout(pad=0.5)
plt.subplot(234)
main_df["SimpleOrderPropensity"].plot.box()
plt.tight_layout(pad=0.5)
plt.subplot(235)
main_df["CollectOrderPropensity"].plot.box()
plt.tight_layout(pad=0.5)
```



The box plots above show us that the data is imbalanced severely, such that the quartiles are all of the same value and that the data has a lot of outliers like discussed earlier. The box plots depict the range and quartile values on the axis and thus tells us the range of our variables as well.

5.3 Sample Exploratory Data Analysis

Sampled data's analysis follows similar steps explained in the previous subsection, to show comparison of both the data sets. The comparison enables us to understand how much similar the sample is to the whole data. The data in the sample is randomly chosen from the original dataset to ensure similarly distributed data instances to ensure the machine learning models makes use of all the features.

	DemandUnits	OrderedUnits	CollectedUnits	SimpleOrderPropensity	CollectOrderPropensity
count	1000000.00	1000000.00	1000000.00	1000000.00	1000000.00
mean	1.09	0.87	0.67	0.80	0.62
std	0.31	0.47	0.54	0.39	0.48
min	1.00	0.00	0.00	0.00	0.00
25%	1.00	1.00	0.00	1.00	0.00
50%	1.00	1.00	1.00	1.00	1.00
75%	1.00	1.00	1.00	1.00	1.00
max	3.00	2.00	2.00	1.00	1.00

The sampled data has a lower mean and deviation for the variables as we got rid of the outliers (elaborated in the next section). The data has 56,411 different types of products from a total of 65,160 products and has 1243 different locations out of the original 1247 locations.

```
28152    364
61980    336
69395    335
55943    320
22927    315
        ...
77405      1
38404      1
66817      1
69497      1
37962      1
Name: ProductKey, Length: 56411, dtype: int64
```

```
25194    4537
72448    3872
26800    3620
67378    3550
47636    3401
         ...
1873        1
12906       1
14920       1
61254       1
58455       1
Name: LocationKey, Length: 1243, dtype: int64
```

The data has a few lesser value counts for HierarchyLevel1 and HierarchyLevel2 but similar number of LocationType1 and LocationType2.

```
len(main_df["HierarchyLevel1"].value_counts())
```

```
180
```

+ Code + Markdown

```
len(main_df["HierarchyLevel2"].value_counts())
```

```
26
```

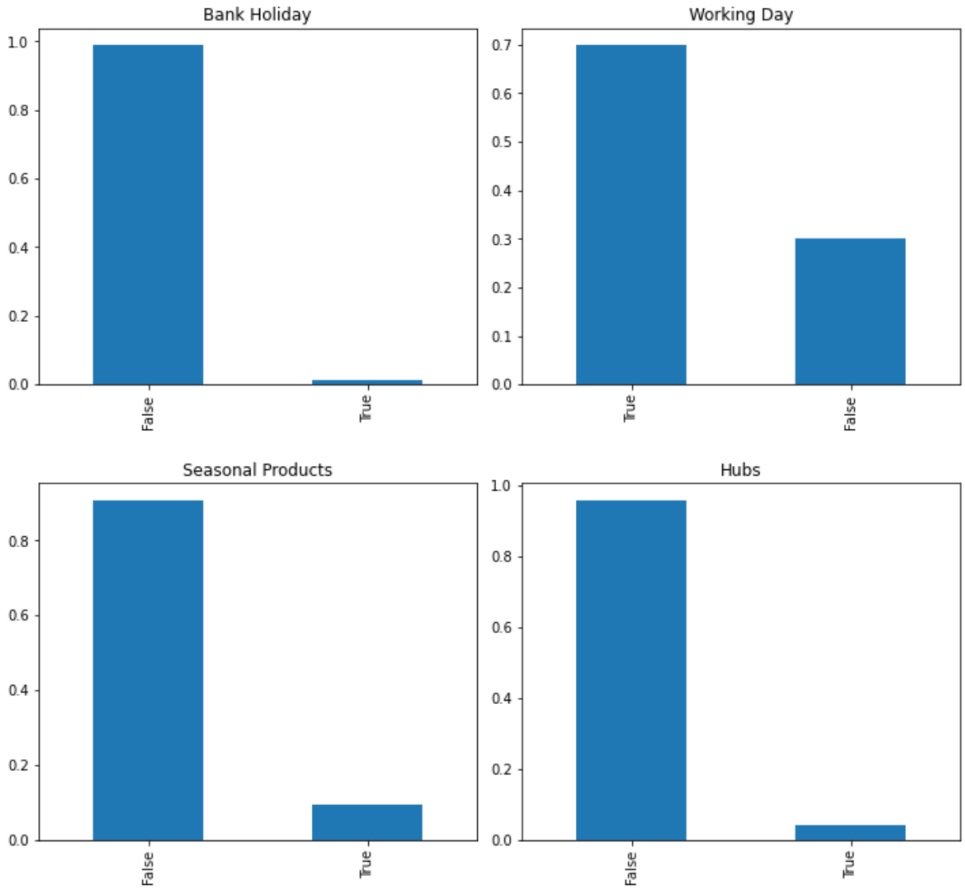
```
len(main_df["LocationType1"].value_counts())
```

```
7
```

```
len(main_df["LocationType2"].value_counts())
```

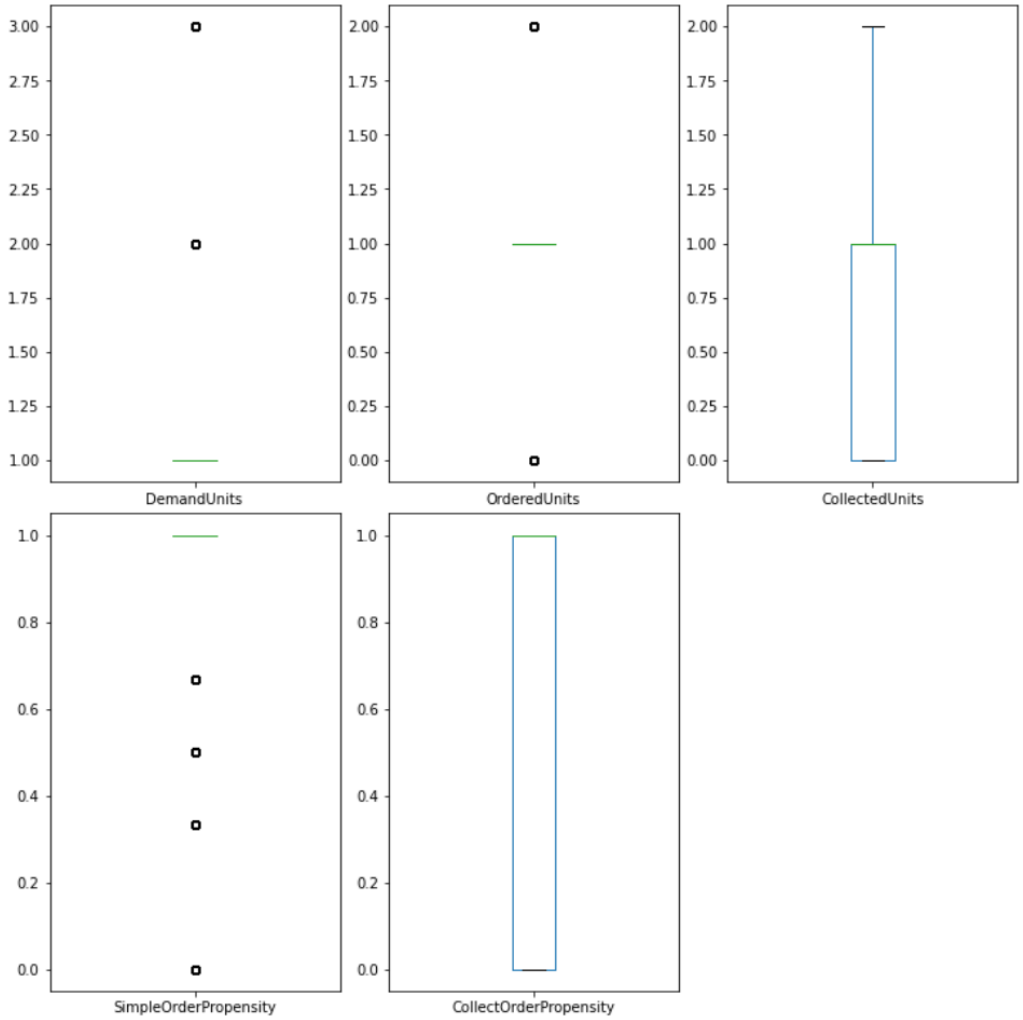
```
3
```

5.3.1 Graph Plots for Nominal Variables



The distributions seem to be exactly the same as the original data with similar percentage of True or False value as in previous EDA.

5.3.2 Graph Plots for Continuous Variables



The sample data has been dealt with outliers hence there seems to be lesser/ no abnormal values, further elaborated in the next section. It can be observed in the graphs above that the values are not absolutely continuous instead many instances are of the same values e.g. Demand Units are only either 1, 2 or 3, Ordered Units is either 0, 1 or 2 and similarly Simple Order Propensity is 0, 0.33, 0.5, 0.67 and 1. This suggests that classification models such as Logistic Regression, KNN and Random Forest Classifiers can be applied on the data as well as Regression algorithms due to the nature of the instances in the data.

5.3.3 Comparison

The randomly sampled data and the original data has high similarity which ensures the prediction results to be approximately same as of for the original data when the machine learning models are applied. Furthermore, the models can be experimented with the original dataset to observe changes or fluctuations in the prediction results.

5.4 Data Pre-processing

Firstly the outliers are dealt with, as discussed about them in the previous sub-section. Outliers can make the machine learning models to wrongly estimate the target variables are the values are extreme in the sense of the dataset hence we remove them from the data. Since the dataset is large it had a lot of outliers. To calculate the outlier's exact value Mean and Standard Deviation Method was used $mean + 3std$ (std: Standard Deviation) (Oracle, n.d.). The formula was applied on DemandUnits, OrderedUnits and CollectedUnits variables. Propensity values greater than 1 were also removed as it was not sensible to have instances where orders were greater than the actual demand of products.

The identifying variables and date variable are removed from the data as they should not be used to build a machine learning model. The machine learning models can greatly generalize on the identifiers and overfit on the data. Hence, we remove ProductKey, LocationKey and Date columns. We then also remove the DemandUnits, OrderedUnits and CollectedUnits since we want to predict future data and we would not know Demand, Orders or Collections from future dates as input prior.

It is important to note the data value column in the variable table illustrated in the Dataset description section as that decides how the variable should be dealt with. (Encoding / Scaling). The ordinal data variables are converted to string data type so that they can be encoded using pd.get_dummies() function from pandas library. The function can return a sparse matrix or a normal dataframe based on the hyperparmeter. Machine learning models need data to be numerical values as the algorithmic calculations can only be carried out on numbers e.g. a linear regression algorithm would only be able to find a line of fit for numeric data. We use one hot encoding as the variables do not have any order in terms of their values which means one value is not greater or lesser than the other unlike 2 > 1. If we label encode the variables then the model would perform bad treating the values in order 0:bad; 1:good; 2:better; 3:best and so on. For the continuous numeric variables like (Longitude, Latitude and DemandUnits) we use MinMaxScaler() from sklearn's preprocessing class. It is essential to have all the numerical attributes to have a similar scale for the model to perform optimally. Min max scaling is also known as normalization.

5.5 Hypothesis and Intuitions from Data Analysis

With the help of the data analysis elaborated in the previous subsections, a few hypothesis situations were created. The bar graphs for Bank Holiday, Seasonal Products and Hubs show that the data is inclined towards one value greatly, i.e. approximately only 2% of the data instances are True for Bank Holidays, 12% for Seasonal Products and only 4% for Hubs. This proposes the removal of these features as the values are excessively of one category than the other. This would also prevent the model from overfitting. As stated previously that the HierarchyLevel1 maps many-to-one to HierarchyLevel2 and similarly the LocationType1 can be mapped onto the LocationType2 hence these columns can be merged together reducing the number of features and also retaining the useful

information. Since, the values are categorical in nature it is easy to merge them into two distinct joint columns namely prodHierarchy and locType as shown below.

prodHierarchy	locType
32668-71645	1-57890
32668-71645	1-57890
32668-71645	1-57890
32668-71645	1-57890
32668-71645	1-29524

It is believed that inclusion of demand of the products for the specific dates can increase model accuracy and lower the error, as the propensities are calculated using the demand units. Argos are researching on demand forecast as well. The demand units predicted from the separate machine learning model can be used as an input variable in the propensity prediction model. However the model might perform worse if the demand predictions are not accurate to a certain level. Model development is carried out with more various combinations of features to achieve the best result.

6 MODEL DEVELOPMENT

With the help of Data Analysis we made some intuitions about the data and how it's feature variables should be treated to achieve optimal results. We perform feature selection based on the developed insights of the data to test and observe how inclusion of different feature variable affects the model overall. Below are some of the sets of features we used together in order to create the best possible machine learning model.

6.1 Feature Settings (according to hypothesis)

Different combinations of features are selected:

- **Less Features**: YearWeek, DayOfWeek, IsWorkingDay, locType (LocationType2 - LocationType1), prodHierarchy (HierarchyLevel2 – HierarchyLevel1)
  Number of features: 5
  Type of data: All categorical data, sparse matrix

```
X = pre_df[["YearWeek", "DayOfWeek", "IsWorkingDay", "locType", "prodHierarchy"]]
yS = pre_df["SimpleOrderPropensity"]
yC = pre_df["CollectOrderPropensity"]
```

- **All Features**: YearWeek, DayOfWeek, IsWorkingDay, IsBankHoliday, HierarchyLevel1, HierarchyLevel2, Seasonal, IsHub, LocationType1, LocationType2, Latitude, Longitude

Number of features: 12

Type of data: Categorical (one hot encoded) + Numerical (normalized) = numpy matrix

```
Xall = main_df[["YearWeek", "DayOfWeek", "IsWorkingDay", "IsBankHoliday",
                "HierarchyLevel1", "HierarchyLevel2", "Seasonal", "IsHub",
                "LocationType1", "LocationType2", "Latitude", "Longitude"]].copy()
```

- **All Features including DemandUnits**
  Trying all the data with demand units observing the effect of inclusion of demand units in training data.
- **All Features without Longitude and Latitude**
  Trying all categorical data together. Only using one hot encoded (0, 1) values for model training. Assumption that latitude and longitude would not prove to be useful in context of propensities as people would order-in at shops close to them for convenience.
- **All Features without Longitude and Latitude including DemandUnits**
  Trying previous feature setting but with demand units to observe the impact of demand units with only categorical data.

6.2 Model Settings (Hyperparameter Tuning)

Due to the large volume of data and limitations in computational resources available for the project, the model experimentation has been carried out with random 1,000,000 rows sample from the overall pre-processed data. Different samples deliver different errors due to sampling bias as one sample might not have all the possible training variable values e.g. two samples would have two different range of HierarchialLevel1 values which would then be learnt by the model and in return increase or decrease the error. Hence, one random million row sample is saved into a .csv file and then loaded for all the models and feature settings to fairly judge the models.

The sample data is split into training and test data with a ratio of 80% - 20%. Ratio of 70% - 30% is also used by researchers in many machine learning problems however that sample was large enough hence 200,000 examples of variables were enough to test the model prediction. The data is split using sklearn's train_test_split function. The models are tested with SimpleOrderPropensity as a target label.

- **Linear Regression**: default
- **LASSO**: α = 0.01, 0.0001; max_iter =10000, 100000
- **Ridge:** default; α = 0.01, 0.1, 1, 10, 100
- **Random Forest Regressor:** n_estimators = 5, 10; max_depth = 10, 20; bootstrap = True; n_jobs = -1
- **XGBoost:** default

6.3 Evaluation Metrics

The performance of the models have been quantified with the help of various evaluation metrics, which will compare the predicted values on test data of features to the test data of target variables.

Evaluation Metric	Metric Formula
R^2 (R squared)	$R^2 = 1 - \frac{\sum(y_i - \hat{y})^2}{\sum(y_i - \bar{y})^2}$ $\hat{y}$ – predicted value of y
MSE (Mean Squared Error)	$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2$ $\hat{y}$ – predicted value of y
RMSE (Root Mean Squared Error)	$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2}$ $\hat{y}$ – predicted value of y
MAE (Mean Absolute Error)	$MAE = \frac{1}{N}\sum_{i=1}^{N}\lvert y_i - \hat{y}\rvert$ $\hat{y}$ – predicted value of y

The R squared value shows the accuracy of the model on the train or test data set. R squared has been applied upon test data to get the accuracy or actual predicted values. The closer the value to 1 means the better the model has generalized or learned the data. However too much accuracy can also be a sign of overfitting hence we use other evaluation metrics such as Root Mean Squared Error and Mean Absolute Error that tell the error between the predictions and true values. The lower the values are and closer to zero the better the model performs.

7 RESULTS AND DISCUSSION

This section illustrates the results of different machine learning models and set of features in the form of a table. The set of features and models with their parameters have been described in the previous section. The tables are then explained in each subsection, all of which are summarized in the last subsection (7.6).

7.1 Setting-1 : Less Features

The tables show best values in the columns as underlined and bold. The second best values are only bold.

Model/Setting	R^2 Score	RMSE	MAE
Linear Regression	**<u>0.0976</u>**	**<u>0.3723</u>**	**<u>0.2839</u>**
Ridge	**<u>0. 0976</u>**	**<u>0. 3723</u>**	**0.2840**
Ridge (α=1)	**<u>0. 0976</u>**	**<u>0. 3723</u>**	**0.2840**
Lasso (α=0.01, max_iter=10000)	0.0200	----	----
Lasso (α=0.0001, max_iter=100000)	0.0882	0.3742	0.2890
Random Forest (n_estimators=10, max_depth=10, bootstrap=True, n_jobs=-1)	0.051	0.3817	0.2986
Random Forest (n_estimators=5, max_depth=20, bootstrap=True, n_jobs=-1)	0.051	0.3797	0.2946
XGBoost	**0.0945**	**0.3730**	**0.2876**

" ----- " : models with very low score are not trained

The table above shows that the greatest R^2 values are for Linear and Ridge regression models at 0.0976. XGBoost model outputs a similar but lower value of 0.0945. The R^2 values are really low as 1 being the greatest value meaning model fits the data perfectly and 0 being the worst value meaning model does not fit the data at all. Here Linear, Ridge and XGBoost models fit to the data rather well than the other models, which can be further proved by the RMSE and MAE values as Linear and Ridge models have the lowest RMSE = 0.3723 and the Linear Regression model has the lowest MAE = 0.2839 and Ridge has a slight more of 0.2840. XGBoost have similar but higher error values of 0.3730 and 0.2876.

7.2 Setting-2: All Features

Model/Setting	R^2 Score	RMSE	MAE
Linear Regression	**0.0982**	0.3721	0.2846
Ridge	**0.0981**	0.3721	0.2845
Ridge (α=0.01)	**0.0982**	0.3721	0.2845
Lasso (α=0.01, max_iter=10000)	0.0294	----	----
Lasso (α=0.0001, max_iter=100000)	0.0939	0.3730	0.2871
Random Forest (n_estimators=10, max_depth=10, bootstrap=True, n_jobs=-1)	0.0914	0.3737	0.2866
Random Forest (n_estimators=5, max_depth=20, bootstrap=True, n_jobs=-1)	0.0918	**0.3717**	**0.2808**
XGBoost	**0.142**	**0.3631**	**0.2735**

Linear Regression, Ridge and XGBoost models appear to have the best R^2 scores with XGBoost model having the best R^2 score of 0.142, the Linear and ridge models have 0.0060 lesser score than the best score suggesting that they fit better to the data than the other models. XGBoost RMSE = 0.3631 and MAE = 0.2735 values are the lowest errors amongst all and Random Forest has the second lowest error values even lesser than the Linear and Ridge modles that fit better to the data than Random Forest. This suggest that Random Forest might perform better with more data instances and it would allow the algorithm to generalize better to the data and predict well. However, currently XGBoost model performs the best amongst all the aspects.

7.3 Setting-3: All Features + DemandUnits

Model/Setting	R^2 Score	RMSE	MAE

Linear Regression	**0.1021**	0.3713	0.2820
Ridge	**0.1021**	0.3713	0.2820
Ridge (α=0.01)	**0.1021**	0.3713	0.2820
Lasso (α=0.01, max_iter=10000)	0.0294	----	----
Lasso (α=0.0001, max_iter=100000)	0.0978	0.3722	0.2844
Random Forest (n_estimators=10, max_depth=10, bootstrap=True, n_jobs=-1)	0.094	0.3730	0.2853
Random Forest (n_estimators=5, max_depth=20, bootstrap=True, n_jobs=-1)	0.094	**0.3702**	**0.2779**
XGBoost	**0.148**	**0.3616**	**0.2705**

Similar to the previous results table XGBoost fits the best along with Linear and Ridge regression models however XGBoost predicts with the least errors along with Random Forest model rather than the Linear and Ridge models. This suggest that the Linear and Ridge models might be overfitting slightly to the data as their R^2 scores are better but error values are greater than Random Forest.

7.4 Setting-4: (no Latitude/Longitude) All Features

Model/Setting	R^2 Score	RMSE	MAE
Linear Regression	**0.0967**	**0.3725**	**0.2846**
Ridge	**0.0966**	**0.3725**	**0.2846**
Ridge (α=0.01)	**0.0966**	**0.3725**	**0.2846**
Lasso (α=0.01, max_iter=10000)	0.0294	----	----
Lasso (α=0.0001, max_iter=100000)	0.0924	0.3733	0.2874

Random Forest (n_estimators=10, max_depth=10, bootstrap=True, n_jobs=-1)	0.058	0.3804	0.2964
Random Forest (n_estimators=5, max_depth=20, bootstrap=True, n_jobs=-1)	0.058	0.3783	0.2919
XGBoost	**0.101**	**0.3716**	**0.2849**

In this feature setting the Linear Regression and Ridge models outperform Random Forest model in all the score and error values unlike in previous two settings. The XGBoost model has a better R^2 score and RMSE value which is lesser than Linear and Ridge however they have a slightly lower MAE score than the XGBoost. The linear and ridge models though do not have lower error values than in the previous settings hence it can be assumed that this feature setting is not suitable.

7.5 Setting-5: (no Lat/Long) All Features + DemandUnits

Model/Setting	R^2 Score	RMSE	MAE
Linear Regression	**0.1006**	**0.3716**	**0.2820**
Ridge	**0.1006**	**0.3716**	**0.2821**
Ridge (α=0.01)	**0.1006**	**0.3716**	**0.2821**
Lasso (α=0.01, max_iter=10000)	0.0294	----	----
Lasso (α=0.0001, max_iter=100000)	0.0964	0.3725	0.2846
Random Forest (n_estimators=10, max_depth=10, bootstrap=True, n_jobs=-1)	0.063	0.3793	0.2940
Random Forest (n_estimators=5, max_depth=20,	0.063	0.3769	0.2885

bootstrap=True, n_jobs=-1)			
XGBoost	**0.108**	**0.3702**	**0.2816**

In this feature setting XGBoost perform the best in all the evaluation metrics however just slightly better than the Linear and Ridge models. This feature setting is same as the previous one with inclusion of Demand Units to observe its effect on the models. The model does not perform the best overall or better than All Feature Settings.

7.6 Results Summary

Best result: All Features and All Features + DemandUnits

Features Variables	Models	R^2 Score	RMSE	MAE
All Features	XGBoost	0.142	0.3631	0.2735
All Features including DemandUnits	XGBoost	0.148	0.3616	0.2705

Linear regression performs the best in the first feature setting of Less Features with a RMSE value of 0.3723, however the error decreases when All Features are used. The RMSE value for linear regression decreases to 0.3721 but XGBoost performs the best this time with a RMSE value of 0.3631 and MAE of 0.2735. This scores remains the best score amongst all the feature settings except the inclusion of DemandUnits with All Features setting in which the R^2 value of the model increases to 0.148 from previous best 0.142. The RMSE value decreases to 0.3616 and MAE value decreases to 0.2705. This confirms the hypothesis that using a demand units as an input variable would help the model to perform better. Removing latitude and longitude variables and using all one hot encoded data did not result into a lower error as per hypothesis. The hypothesis developed in the start of model development about training models on less feature variables was not correct at all in fact using all the feature variables decreased the error by approximately 0.01. XGBoost model performed the best as expected due the prior research, elaborated in Literature Review section. The errors are less however it can be decreased further more as observed that the models are not fitting the data well which might be due to the lack of good predicting feature variables.

The RMSE, MAE values for XGBoost model to predict CollectOrderPropensity was higher than that of SimpleOrderPropensity. R^2 score of 0.139 was achieved when the model was trained using CollectOrderPropensity as a target variable. The RMSE = 0.4436 and MAE = 0.4015 was much higher than XGBoost with the same feature set of All Features although both the variables are of the same range and nature. This suggests that the model requires more useful training features and also instances.

8 CRITICAL APPRAISAL

For this project so far machine learning and data science techniques have been applied which has taught us a lot of critical thinking and intuition building. It also taught that experimentation is the key to develop good machine learning models as intuitions and hypothesis can go wrong, and through experimentation different effects of various attributes can be leant which helps in understanding the data better. Understanding of the data is essential in machine learning model development as the model itself has to learn from the data and the quality of the data decides a models capability. Time management and project management skills were learnt through out the project. Below are stated the alternatives of different aspects of the project which can remove some limitations as well leading to a better prediction model.

The project can also be treated as a classification problem as the target variables, with a range of 0 – 1, could have been altered to discrete values using a certain threshold e.g. values less than 0.15 can be changed to 0, values greater than 0.15 and less than 0.35 can be treated as 0.25, similarly 0.35 < value < 0.6 as 0.5, and so on or a similar threshold of values. The current values of the sample target variables already show a similar pattern as shown below.

```
1.000000    788961
0.000000    187958
0.500000     19694
0.666667      1763
0.333333      1624
Name: SimpleOrderPropensity, dtype: int64
```

Artificial Neural Networks are being used to solve many different problems and it is being used in numerous similar cases e.g. sales prediction. It will be worth experimenting the neural networks and their its comparison to the current best predictor (XGBoost) and its error value.

It is also a rational intuition from the research on weather affecting customer behaviour to include weather data like average daily temperature, amount of precipitation for the day and UV index to train the model with the current data. According to the research in the Literature Review section, it can be assumed that the inclusion of certain weather data attributes mentioned above can have a positive impact on the predictions and can help lower model accuracy. The effects of economical attributes such as GDP per capita, inflation rate and life satisfaction on consumer behaviour, psychology and in relation to retail business although have not been thoroughly researched however might prove to be useful to learn further trends in the data.

Due to limitations in computing power and resources it was not possible to train and test the machine learning models on the whole of the data which would have helped in grasping a more accurate overview of the data and its variables. Incremental and out of core learning models can be tested upon the dataset with further research in libraries like vowpal wabbit and crème that support incremental learning in machine learning algorithms.

Advancements in parallel computing can also be made use of to a good extent with subscriptions for computational clusters as a service provided by AWS (Amazon Web Services), Microsoft Azure Machine Learning Platform, Apache Spark clusters by Databricks. The clusters can be used to split model development tasks ultimately providing the capability to work on large datasets. A standalone system with a good amount of ram up to 32gb can also be considered for training and testing the whole dataset with more feature variables.

9 CONCLUSION

The report presents a prediction model for customer order-in and collection propensities, it compares different supervised regression models and discusses the effects of different attribute combinations on model accuracy. The data is huge to deal with hence a sample of random variables is extracted from the data to conduct model experimentation and development. The sample closely relates to the whole dataset as observed through exploratory analysis. One of the main problems faced with the project was to deal with such a large data whilst having limited memory space. The problem was tackled partially by sampling the data and using Kaggle notebook kernel equipped with free 16gh memory space. The impact of weather on similar types of problems are researched upon and future work is also discussed upon in Critical Appraisal section.

It is concluded that XGBoost model performs the best when using all the feature variables, resulting with RMSE of 0.3631 however Random Forest Regressor performs second best with RMSE of 0.3717. Linear and Ridge models have good R^2 however not better than XGBoost. Including the demand units variable as discussed in section 5.5 and 6.1 decreased the current best RMSE error by 0.0015 and MAE by 0.0030. Preliminary study has also been carried out on Artificial Neural Networks and it was found out that neural networks perform worse than all the other models. The XGBoost model performs worse and has a higher error and considerably lower R^2. The model's RMSE is increased by 0.0805 and MAE by 0.1280 while using all the features (Setting-2).

10 REFERENCES

IBM Cloud Education, 2020. *Unsupervised Learning.* [Online]
Available at: https://www.ibm.com/cloud/learn/unsupervised-learning
[Accessed 29 10 2020].

Anderson, E. T., Hansen, K. & Simester, D., 2009. The Option Value of Returns: Theory and Empirical Evidence. *Marketing Science,* 28(3), pp. 405-423.

Argos, 2019. *Great products and services.* [Online]
Available at: https://www.about.sainsburys.co.uk/great-products-and-services/argos
[Accessed 28 10 2020].

Badorf, F. & Hoberg, K., 2020. The impact of daily weather on retail sales: An empirical study in brick-and-mortar stores. *Journal of Retailing and Consumer Services,* Volume 52.

Barga, R., Fontama, V. & Tok, W. H., 2015. *Predictive Analytics with Microsoft Azure Machine Learning.* 2 ed. Berkeley: Apress.

Bertsimas, D., O'Hair, A., Relyea, S. & Silberholz, J., 2016. An analytics approach to designing combination chemotherapy regimens for cancer. *Management Science,* 62(5), pp. 1511-1531.

Bodjov, Y., 2018. *Machine learning in risk forecasting and its application in low volatility strategy.* [Online]
Available at: https://www.tdaminstitutional.com/tmi/pdfs/machlearn0918_E.pdf
[Accessed 20 4 2021].

Charnes, A., Cooper, W. W. & Sueyoshi, T., 1986. Least squares/ridge regression and goal programming/constrained regression alternatives. *European Journal of Operational Research,* 27(2), pp. 146-157.

Chen, T. & Guestrin, C., 2016. *XGBoost: A Scalable Tree Boosting System.* s.l., s.n., pp. 785-794.

Coad, A., 2019. *How Sainsbury's is generating new insights into how the world eats.* [Online]
Available at: https://cloud.google.com/blog/topics/customers/how-sainsburys-is-generating-new-insights-into-how-the-world-eats
[Accessed 29 10 2020].

Dask.org, 2021. *Dask.* [Online]
Available at: https://docs.dask.org/en/latest/
[Accessed 25 4 2021].

Donald, G. C. M. & Schwing, R. C., 1973. Instabilities of Regression Estimates Relating Air Pollution to Mortality. *Technometrics,* 15(3), pp. 463-481.

Fan, C., Cui, Z. & Zhong, X., 2018. *House Prices Prediction with Machine Learning Algorithms.* New York, Association for Computing Machinery, p. 6–10.

Forbes, 2017. *How AI and machine learning are helping drive the GE digital transformation.* [Online]
Available at: https://www.forbes.com/sites/ciocentral/2017/06/07/how-ai-and-machine-learning-are-helping-drive-the-ge-digital-transformation/#4f3464321686
[Accessed 21 4 2021].

Freund, Y. & Schapire, R. E., 1999. A Short Introduction to Boosting. *Journal of Japanese Society for Artificial Intelligence,* 14(5), pp. 771-780.

Friedman, J. H., 2001. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics,* 29(5), pp. 1189-1232.

GOV.UK, 2021. *UK bank holidays.* [Online]
Available at: https://www.gov.uk/bank-holidays
[Accessed 24 4 2021].

Grosch, K., 2018. *John Deere – Bringing AI to Agriculture,* s.l.: HBS Digital Initiative.

Hess, J. D. & Mayhew, G. E., 1997. Modeling merchandise returns in direct marketing. *Journal of Direct Marketing,* 11(2), pp. 20-35.

He, X. et al., 2014. *Practical Lessons from Predicting Clicks on Ads at Facebook.* s.l., s.n., pp. 1-9.

Hoerl, A. E. & Kennard, R. W., 1970. Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics,* 12(1), pp. 55-67.

IDC, 2018. *Worldwide spending on cognitive and artificial intelligence systems forecast to reach $77.6 billion in 2022, according to new IDC spending guide.* [Online]
Available at: https://www.idc.com/getdoc.jsp?containerId=prUS44291818
[Accessed 23 4 2021].

Indeed.com, 2019. *The best jobs in the U.S.: 2019.* [Online]
Available at: http://blog.indeed.com/2019/03/14/best-jobs-2019/
[Accessed 23 4 2021].

Janakiraman, N., Syrdal, H. A. & Freling, R., 2016. The Effect of Return Policy Leniency on Consumer Purchase and Return Decisions: A Meta-analytic Review. *Journal of Retailing,* 92(2), pp. 226-235.

Kang, S. H., Jiang, Z., Lee, Y. & Yoon, S.-M., 2010. Weather effects on the returns and volatility of the Shanghai stock market. *Physica A: Statistical Mechanics and its Applications,* 389(1), pp. 91-99.

Li, P., 2010. *Robust logitboost and adaptive base class (ABC) logitboost.* Catalina Island, AUAI Press, pp. 302-311.

Manyika, J. et al., 2011. *Big data: The next frontier for innovation, competition and productivity.* [Online]
Available at:
https://www.mckinsey.com/~/media/McKinsey/Business%20Functions/McKinsey%20Digital/Our%20Insights/Big%20data%20The%20next%20frontier%20for%20innovation/MGI_big_data_exec_summary.ashx
[Accessed 28 10 2020].

Martinez, A. et al., 2020. A machine learning framework for customer purchase prediction in the non-contractual setting. *European Journal of Operational Research,* 281(3), pp. 588-596.

Ma, S., Fildes, R. & Huang, T., 2016. Demand forecasting with high dimensional data: The case of SKU retail sales forecasting with intra- and inter-category promotional information. *European Journal of Operational Research,* 249(1), pp. 245-257.

Matplotlib.org, 2021. *Matplotlib.* [Online]
Available at: https://matplotlib.org/
[Accessed 25 4 2021].

McDowell, H., 2018. *JP Morgan trading team develops machine learning model for equity desk.* [Online]
Available at: https://www.thetradenews.com/jp-morgans-trading-team-develop-machine-learning-model-equity-desk/
[Accessed 20 4 2021].

Mittelstadt., B. D. et al., 2016. The ethics of algorithms: Mapping the debate. *Big Data and Society,* 3(2), pp. 1-21.

Murphy, K. P., 2012. *Machine Learning: A Probabilistic Perspective.* Palo Alto: The MIT Press.

Murray, K. B., Muro, F. D., Finn, A. & Leszczyc, P. P., 2010. The effect of weather on customer spending. *Journal of Retailing and Consumer Services,* 17(6), pp. 512-520.

NumPy, 2021. *NumPy v1.20.0.* [Online]
Available at: https://numpy.org/
[Accessed 25 4 2021].

O'Reilly Media, 2018. *The state of machine learning adoption in the enterprise.* [Online]
Available at: https://www.oreilly.com/data/free/state-of-machine-learning-adoption-in-the-enterprise.csp
[Accessed 23 4 2021].

Oracle, n.d. *Outlier Detection Methods.* [Online]
Available at:
https://docs.oracle.com/cd/E17236_01/epm.1112/cb_statistical/frameset.htm?ch07s02s10s01.html
[Accessed 24 4 2021].

Pow, N., Janulewicz, E. & Liu, L., 2014. *Applied Machine Learning Project 4 Prediction of real estate property prices in Montréal,* s.l.: McGill University.

pydata.org, 2021. *pandas.* [Online]
Available at: https://pandas.pydata.org/
[Accessed 25 4 2021].

Ranstam, J. & Cook, J. A., 2018. LASSO regression. *British Journal of Surgery,* 105(10), p. 1348.

Rincon-Patino, J., Lasso, E. & Corrales, J. C., 2018. Estimating Avocado Sales Using Machine Learning Algorithms and Weather Data. *Sustainability,* 10(10).

Santis, R. B. d., Aguiar, E. P. d. & Goliatt, L., 2017. *Predicting material backorders in inventory management using machine learning.* Arequipa, IEEE.

sklearn.org, 2021. *scikit-learn Machine Learning in Python.* [Online]
Available at: https://sklearn.org/
[Accessed 25 4 2021].

Tibshirani, R., 1996. Regression Shrinkage and Selection Via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological),* 58(1), pp. 267-288.

Vittinghoff, E., Glidden, D. V., Shiboski, S. C. & McCulloch, C. E., 2005. Linear Regression. In: *Regression Methods in Biostatistics: Linear, Logistic, Survival, and Repeated Measures Models.* New York: Springer, pp. 69-131.

Walshaw, C., 2020. A Tale of Two KTPs – An Academic Perspective. *Mathematics Today,* pp. 157-158.

APPENDIX A - SOURCE CODE

MODEL DEVELOPMENT CODE:

Importing libraries

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
from dask import dataframe as dd

%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
import gc

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
from sklearn.model_selection import cross_val_score
import xgboost as xgb
```

Merging datasets

```
main_df = dd.merge(data, date_data, left_on='Date', right_on='DateKey', how='left')
```

```
main_df = dd.merge( main_df , prod_data, left_on='ProductKey', right_on='ProductKey', how='left')
```

```
main_df = dd.merge( main_df , loc_data, left_on='LocationKey', right_on='LocationKey', how='left')
```

```
main_df =  main_df .drop(["DateKey", "CatEdition", "Supplier", "DIorDOM",
                          "Region", "UpstreamLocKey", "StockPolicy", "ShopFormat"], axis=1)
```

Calculating propensities and using dask compute method to carry out the transformation.

```
main_df["SimpleOrderPropensity"] = ( main_df.OrderedUnits / main_df .DemandUnits)
```

+ Code + Markdown

```
main_df["CollectOrderPropensity"] = ( main_df.CollectedUnits /  main_df.DemandUnits)
```

+ Code + Markdown

```
main_df = main_df.compute()
```

```
del data, loc_data, prod_data, date_data
gc.collect()
```

Removing outliers

```
outlier = main_df['DemandUnits'].mean() + (3*main_df['DemandUnits'].std())
main_df = main_df.drop((main_df[main_df['DemandUnits'] >= outlier]).index, 0)

outlier = main_df['OrderedUnits'].mean() + (3*main_df['OrderedUnits'].std())
main_df = main_df.drop(( main_df [main_df['OrderedUnits'] >= outlier]).index, 0)

outlier = main_df['CollectedUnits'].mean() + (3*main_df['CollectedUnits'].std())
main_df = main_df.drop((main_df[main_df['CollectedUnits'] >= outlier]).index, 0)

main_df = main_df.drop((main_df[main_df['SimpleOrderPropensity'] > 1]).index, 0)
main_df = main_df.drop((main_df[main_df['CollectOrderPropensity'] > 1]).index, 0)
```

Getting number of lines of the whole dataset after removing outlier

```
lines = main_df.shape[0]
```

Selecting 1,000,000 rows sample

```
skiplines = np.random.choice(np.arange(1, lines), size=lines-1000000, replace=False)

#sort the list
skiplines=np.sort(skiplines)
```

```
main_df = pd.read_csv("../input/main1m/main1m.csv", skiprows=skiplines, nrows=1000000) #
```

Converting categorical features to string for one hot encoding

```
main_df['YearWeek'] = main_df["YearWeek"].astype(str)
main_df['DayOfWeek'] = main_df["DayOfWeek"].astype(str)
main_df['IsBankHoliday'] = main_df["IsBankHoliday"].astype(str)
main_df['IsWorkingDay'] = main_df["IsWorkingDay"].astype(str)
main_df['HierarchyLevel1'] = main_df["HierarchyLevel1"].astype(str)
main_df['HierarchyLevel2'] = main_df["HierarchyLevel2"].astype(str)
main_df['Seasonal'] = main_df["Seasonal"].astype(str)
main_df['IsHub'] = main_df["IsHub"].astype(str)
main_df['LocationType1'] = main_df["LocationType1"].astype(str)
main_df['LocationType2'] = main_df["LocationType2"].astype(str)
```

Separating Feature and Target variable

- Xless; Less feature setting from Model Development
- Xall: All feature setting from Model Development

```
Xall = pd.DataFrame()
# Xless = pd.DataFrame()
Xall = main_df[["YearWeek", "DayOfWeek", "IsWorkingDay", "IsBankHoliday",
                "HierarchyLevel1", "HierarchyLevel2", "Seasonal", "IsHub",
                "LocationType1", "LocationType2", "Latitude", "Longitude"]].copy()

# Xless = main_df[["YearWeek", "DayOfWeek", "IsWorkingDay"]].copy()
# Xless['prodHierarchy'] = main_df["HierarchyLevel2"] + "-" + main_df["HierarchyLevel1"]
# Xless['locType'] = main_df["LocationType2"] + "-" + main_df["LocationType1"]

yS = main_df["SimpleOrderPropensity"]
yC = main_df["CollectOrderPropensity"]
```

Feature Transformation and train test split

- One hot encoding applied to Xcat using od.get_dummies
- MinMaxScaling (Normalization of continuous numeric variables)

```
catFeat = ["YearWeek", "DayOfWeek", "IsWorkingDay", "IsBankHoliday", "HierarchyLevel1", "HierarchyLevel2",
           "Seasonal", "IsHub", "LocationType1", "LocationType2"]

numFeat = ["Latitude", "Longitude"]

numTransf = MinMaxScaler()
Xnum = numTransf.fit_transform(Xall[numFeat])

Xcat = pd.get_dummies(Xall[catFeat], sparse = False)

Xall = np.concatenate((Xnum, Xcat), axis=1)
```

+ Code + Markdown

```
X_train, X_test, yS_train, yS_test = train_test_split(Xall, yS, test_size = 0.2, random_state = 0)
```

Linear Regression: model fit, prediction and print evaluation metric values

```
linReg = LinearRegression().fit(X_train, yS_train)
print("Training set score: {:.4f}".format(linReg.score(X_train, yS_train)))
print("Test set score: {:.4f}".format(linReg.score(X_test, yS_test)))
```

```
y_pred = linReg.predict(X_test)
rmse = np.sqrt(mean_squared_error(yS_test, y_pred))
print("Root Mean Squared Error: {:.4f}".format(rmse))
mae = mean_absolute_error(yS_test, y_pred)
print('Mean Absolute Error: ', mae)
```

Ridge model training and prediction

```
best_score = 0
best_alpha = 0
for alpha in [0.01, 0.1, 1, 10, 100]:
    ridge = Ridge(alpha=alpha).fit(X_train, yS_train)
    testscore = ridge.score(X_test, yS_test)
    if testscore > best_score:
        best_score = testscore
        best_alpha = alpha
print('Best score: {:.4f}'.format(best_score))
print('Best alpha: {:.2f}'.format(best_alpha))
```

+ Code + Markdown

```
ridge = Ridge(alpha=best_alpha).fit(X_train, yS_train)
```

```
y_pred = ridge.predict(X_test)
rmse = np.sqrt(mean_squared_error(yS_test, y_pred))
print("Root Mean Squared Error: {:.4f}".format(rmse))
mae = mean_absolute_error(yS_test, y_pred)
print('Mean Absolute Error: ', mae)
```

Lasso model training and prediction

```
lasso1 = Lasso(alpha=0.01, max_iter=100000).fit(X_train, yS_train)
print("Training set score: {:.4f}".format(lasso1.score(X_train, yS_train)))
print("Test set score: {:.4f}".format(lasso1.score(X_test, yS_test)))
print("Number of features used: {}".format(np.sum(lasso1.coef_ != 0)))
```

```
lasso2 = Lasso(alpha=0.0001, max_iter=100000).fit(X_train, yS_train)
print("Training set score: {:.4f}".format(lasso2.score(X_train, yS_train)))
print("Test set score: {:.4f}".format(lasso2.score(X_test, yS_test)))
print("Number of features used: {}".format(np.sum(lasso2.coef_ != 0)))
```

```
y_pred = lasso2.predict(X_test)
rmse = np.sqrt(mean_squared_error(yS_test, y_pred))
print("Root Mean Squared Error: {:.4f}".format(rmse))
mae = mean_absolute_error(yS_test, y_pred)
print("Mean Absolute Error: ", mae)
```

Random Forest Regressor model training and testing

```
forest = RandomForestRegressor(n_estimators=10, max_depth=10, bootstrap=True, n_jobs=-1)
forest.fit(X_train, yS_train)
print("Accuracy on training set: {:.3f}".format(forest.score(X_train, yS_train)))
print("Accuracy on test set: {:.3f}".format(forest.score(X_test, yS_test)))
```

```
y_pred = forest.predict(X_test)
rmse = np.sqrt(mean_squared_error(yS_test, y_pred))
print("Root Mean Squared Error: {:.4f}".format(rmse))
mae = mean_absolute_error(yS_test, y_pred)
print('Mean Absolute Error: ', mae)
```

```
forest1 = RandomForestRegressor(n_estimators=5, max_depth=20, bootstrap=True, n_jobs=-1)
forest1.fit(X_train, yS_train)
print("Accuracy on training set: {:.3f}".format(forest.score(X_train, yS_train)))
print("Accuracy on test set: {:.3f}".format(forest.score(X_test, yS_test)))
```

```
y_pred = forest1.predict(X_test)
rmse = np.sqrt(mean_squared_error(yS_test, y_pred))
print("Root Mean Squared Error: {:.4f}".format(rmse))
mae = mean_absolute_error(yS_test, y_pred)
print('Mean Absolute Error: ', mae)
```

XGBoost training and testing

```
import xgboost
xgb_reg = xgboost.XGBRegressor()
xgb_reg.fit(X_train, yC_train)
y_pred = xgb_reg.predict(X_test)
```

```
print("Accuracy on test set: {:.3f}".format(xgb_reg.score(X_test, yC_test)))
```

+ Code + Markdown

```
rmse = np.sqrt(mean_squared_error(yC_test, y_pred))
print("Root Mean Squared Error: {:.4f}".format(rmse))
mae = mean_absolute_error(yC_test, y_pred)
print('Mean Absolute Error: ', mae)
```

EXPLORATORY DATA ANALYSIS CODE (Local Jupyter Notebook):

Exploratory Data Analysis

Quick look at the Data

```
import pandas as pd
import matplotlib as mpl
#mpl.rcParams['agg.path.chunksize'] = 10000
import matplotlib.pyplot as plt
import seaborn as sns
import os
path = os.getcwd()
```

Loading data into "main_df" variable

```
#loc_data = pd.read_csv('03_LocationMaster.csv')
# prod_data = pd.read_csv(r'C:\Users\sk166\02_ProductMaster.csv')
# date_data = pd.read_csv(r'C:\Users\sk166\01_CalendarMaster.csv')
#main_df = pd.read_csv(r'C:\Users\sk166\09_Project3.csv')
main_df = pd.read_csv(r'C:\Users\sk166\MainDataFrame.csv')
```

Outputting first and last 5 rows to get fimiliar with data.
Data shows 20 columns with 43,016,599 rows.

```
main_df.head(5)
```

3]:

	Date	ProductKey	LocationKey	DemandUnits	OrderedUnits	CollectedUnits	YearWeek	DayOfWeek	IsBankHoliday	IsWorkingDay	HierarchyLevel1
0	20180828	42972	3260	2	2	2	201835	3	False	True	71645
1	20180820	42972	23485	1	0	0	201834	2	False	True	71645
2	20180826	42972	23485	1	0	0	201835	1	False	False	71645
3	20180827	42972	23485	1	0	0	201835	2	True	False	71645
4	20170815	42972	70984	2	2	2	201733	3	False	True	71645

```
main_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 43016599 entries, 0 to 43016598
Data columns (total 20 columns):
 #   Column                 Dtype
---  ------                 -----
 0   Date                   int64
 1   ProductKey             int64
 2   LocationKey            int64
 3   DemandUnits            int64
 4   OrderedUnits           int64
 5   CollectedUnits         int64
 6   YearWeek               int64
 7   DayOfWeek              int64
 8   IsBankHoliday          bool
 9   IsWorkingDay           bool
 10  HierarchyLevel1        int64
 11  HierarchyLevel2        int64
 12  Seasonal               bool
 13  IsHub                  bool
 14  LocationType1          int64
 15  LocationType2          int64
 16  Latitude               float64
 17  Longitude              float64
 18  SimpleOrderPropensity  float64
 19  CollectOrderPropensity float64
dtypes: bool(4), float64(4), int64(12)
memory usage: 5.3 GB
```

```
corr = main_df.corr()
corr["SimpleOrderPropensity"].sort_values(ascending=False)
```

```
corr["CollectOrderPropensity"].sort_values(ascending=False)
```

Basic statistics of products demand, order, collection and propensities...

```
pd.set_option('display.float_format', lambda x: '%.2f' % x)
main_df.loc[:, ["DemandUnits", "OrderedUnits", "CollectedUnits",
                "SimpleOrderPropensity", "CollectOrderPropensity"]].describe()
```

8]:

	DemandUnits	OrderedUnits	CollectedUnits	SimpleOrderPropensity	CollectOrderPropensity
count	43016599.00	43016599.00	43016599.00	43016599.00	43016599.00
mean	1.14	0.91	0.69	0.80	0.62
std	0.70	0.78	0.65	0.39	0.48
min	1.00	0.00	0.00	0.00	0.00
25%	1.00	1.00	0.00	1.00	0.00
50%	1.00	1.00	1.00	1.00	1.00
75%	1.00	1.00	1.00	1.00	1.00
max	580.00	580.00	294.00	4.00	3.33

It can be seen that average (mean) deamand of units is greater than average orders, and that greater than average units collection.

There have been 65,160 different type of products in Argos inventory being sold across 1,247 stores in 3 years time span!

```
main_df.ProductKey.value_counts()
```

9]:

```
28152    16900
63886    15916
61980    15879
55943    15516
42283    15190
         ...
72782        1
1932         1
34714        1
67495        1
76894        1
Name: ProductKey, Length: 65160, dtype: int64
```

```
main_df.LocationKey.value_counts()
```

0]:

```
25194    195916
72448    166560
26800    155710
67378    152407
39596    146126
          ...
21726        21
58455        20
14920        15
60269        15
1873         11
Name: LocationKey, Length: 1247, dtype: int64
```

```
len(main_df["HierarchyLevel1"].value_counts())
```

```
]: 181
```

HierarchyLevel1 maps many-to-one to HierarchyLevel2. Examples of this hierarchy level include Household Electricals, Technology, Furniture

```
len(main_df["HierarchyLevel2"].value_counts())
```

```
]: 26
```

Classifies locations by their shop-front type. Examples include High Street, Retail Park, Within Supermarket

```
len(main_df["LocationType1"].value_counts())
```

```
]: 7
```

Classifies locations by their surrounding area. Examples include Town Centre, Rural, Destination Shopping Centre

```
len(main_df["LocationType2"].value_counts())
```

```
]: 3
```

Univariate Analysis

Graph Plots for Categorical Variables

Each date has atleast a minimum (1 unit) demand for specific product even if amount of order or collection is zero.

```
plt.subplot(321)
main_df.IsBankHoliday.value_counts(normalize=True).plot(kind="bar", title="Bank Holiday", figsize=(10,13))
plt.tight_layout(pad=1)
plt.subplot(322)
main_df.IsWorkingDay.value_counts(normalize=True).plot(kind="bar", title="Working Day")
plt.tight_layout(pad=1)
plt.subplot(323)
main_df.Seasonal.value_counts(normalize=True).plot(kind="bar", title="Seasonal Products")
plt.tight_layout(pad=1)
plt.subplot(324)
main_df.IsHub.value_counts(normalize=True).plot(kind="bar", title="Hubs")
plt.tight_layout(pad=1)
```

Graph Plots for Products

```
plt.subplot(221)
main_df.DayOfWeek.value_counts(normalize=True).plot(kind="bar",title="Daily Demand Ratio",figsize=(11,13))
plt.tight_layout(pad=1)
plt.subplot(222)
main_df.groupby("ProductKey").DemandUnits.sum().sort_values(ascending=False).head(10).plot(
    kind="bar",title="Top 10 Products by Demand (Overall)")
plt.tight_layout(pad=1)
plt.subplot(223)
main_df.groupby("ProductKey").OrderedUnits.sum().sort_values(ascending=False).head(10).plot(
    kind="bar",title="Top 10 Products by Orders (Overall)")
plt.tight_layout(pad=1)
plt.subplot(224)
main_df.groupby("ProductKey").CollectedUnits.sum().sort_values(ascending=False).head(10).plot(
    kind="bar",title="Top 10 Products by Collection (Overall)")
plt.tight_layout(pad=1)
```

```
plt.subplot(221)
main_df.groupby("LocationKey").OrderedUnits.sum().sort_values(ascending = False).head(10).plot(
    kind="bar", title="Top 10 Store Locations by Orders (Overall)", figsize=(11,13))
plt.tight_layout(pad=1)
plt.subplot(222)
main_df.LocationType2.value_counts(normalize = True).plot(
    kind="bar", title="Town Centre vs Rural vs Destination Shopping Centre")
plt.tight_layout(pad=1)
plt.subplot(223)
main_df.LocationType1.value_counts(normalize = True).plot(
    kind="bar", title="Location Type .. High Street, Retail Park ..")
plt.tight_layout(pad=1)
```

Graph plots for location specific data

```
plt.subplot(221)
main_df.groupby("LocationKey").OrderedUnits.sum().sort_values(ascending = False).head(10).plot(
    kind="bar", title="Top 10 Store Locations by Orders (Overall)", figsize=(11,13))
plt.tight_layout(pad=1)
plt.subplot(222)
main_df.LocationType2.value_counts(normalize = True).plot(
    kind="bar", title="Town Centre vs Rural vs Destination Shopping Centre")
plt.tight_layout(pad=1)
plt.subplot(223)
main_df.LocationType1.value_counts(normalize = True).plot(
    kind="bar", title="Location Type .. High Street, Retail Park ..")
plt.tight_layout(pad=1)
```

Graph Plots for Continious Variables

```
plt.subplot(231)
main_df["DemandUnits"].plot.box(figsize=(10,10))
plt.tight_layout(pad=0.5)
plt.subplot(232)
main_df["OrderedUnits"].plot.box()
plt.tight_layout(pad=0.5)
plt.subplot(233)
main_df["CollectedUnits"].plot.box()
plt.tight_layout(pad=0.5)
# main_df.boxplot(column=['DemandUnits', 'OrderedUnits', 'CollectedUnits'])
plt.subplot(234)
main_df["SimpleOrderPropensity"].plot.box()
plt.tight_layout(pad=0.5)
plt.subplot(235)
main_df["CollectOrderPropensity"].plot.box()
plt.tight_layout(pad=0.5)
```

Target Variable value counts

```
main_df.SimpleOrderPropensity.value_counts()
```

```
]: 1.000000    33970101
   0.000000     7985644
   0.500000      857062
   0.666667       79771
   0.333333       69325
                  ...
   0.318182           1
   0.055556           1
   0.931034           1
   0.140351           1
   0.989362           1
   Name: SimpleOrderPropensity, Length: 236, dtype: int64
```

```
main_df.CollectOrderPropensity.value_counts()
```